# HashMap with Open Addressing & Tombstones

Open Addressing, Linear Probing, Tombstones, Automatic Resizing · Mentor-Led Lab

Developed by TalenciaGlobal

**Difficulty** ★★★★ Advanced

## Problem

Design a MyHashMap that maps integer keys to integer values with open addressing and linear probing - the value-carrying cousin of the previous lab. Keys and values are kept in parallel arrays; a deleted key becomes a tombstone so probe sequences survive.

Support put(key, value) (insert or update), get(key) (the value or -1), and remove(key). Track size as the count of real entries (excluding tombstones), and resize automatically - doubling the capacity and re-inserting only valid keys - when the load factor passes 0.7.

Learning objectives: see how a HashMap stores keys and values together under open addressing, support deletion with tombstones, manage the load factor with resizing, and understand the foundation this lays for Robin Hood and concurrent hash maps.

## Operations & Rules

- PUT key value - insert the pair, or update the value if the key exists.
- GET key - output the value, or -1 if the key is absent.
- REMOVE key - mark the key's slot as a tombstone if present.
- Parallel arrays hold keys and values; the key array uses empty (-1) and tombstone (-2) sentinels. Linear probing visits (hash + i) mod capacity.
- Load factor: resize to double the capacity whenever size / capacity exceeds 0.7 after an insertion; size counts real entries only.

## Input / Output Format

**Input:** Line 1: the initial capacity. Each later line: PUT key value, GET key, or REMOVE key (read until end of input).

**Output:** One line per GET (its result), then Size and Capacity of the final table.

## Examples

### Example 1

**Input:** cap=4; put(1,100); put(5,200); get(1); remove(1); get(1); put(1,300); get(1)

**Output:**

```
100
-1
300
```

**Explanation:** 5 collides with 1 and probes to the next slot; after removing 1 a tombstone holds the slot, and re-putting 1 reuses that tombstone with the new value 300.

### Example 2

**Input:** cap=4; put(2,20); get(2); put(2,25); get(2)

**Output:**

```
20
25
```

**Explanation:** The second put updates the existing key in place.

## Constraints

- 0 <= key, value <= 10^6; up to 10,000 operations.
- Open addressing only - no built-in map types.
- Tombstones for deletion; resize keeps the load factor <= 0.7; size excludes tombstones.

## Approach

Keep parallel keys and values arrays with empty (-1) and tombstone (-2) markers in the key array. The hash is key mod capacity; linear probing scans (hash + i) mod capacity.

put probes for the key: update the value if found; remember the first tombstone; on reaching an empty slot insert at the remembered tombstone (reuse) or the empty slot, and increment size. get stops at the first empty slot returning -1, skipping tombstones; remove writes a tombstone and decrements size.

When size / capacity exceeds 0.7, resize: double the capacity, reset both arrays, and re-put only the valid (key, value) pairs so tombstones are cleared away.

Extensions: the same skeleton supports quadratic probing or double hashing by changing the probe formula, and dropping the value array turns it straight back into the HashSet of the previous lab.

## Solution

*A complete, tested reference solution in each language. All three read the same input and produce identical output.*

### Python

```
# Lab 3 - MyHashMap with Open Addressing (linear probing + tombstones + resizing)
import sys
```

```
EMPTY = -1
TOMB = -2

class MyHashMap:
    def __init__(self, capacity=16):
        self.capacity = capacity
        self.size = 0
        self.keys = [EMPTY] * capacity
        self.values = [0] * capacity

    def _hash(self, key):
        return key % self.capacity

    def _probe(self, key, i):
        return (self._hash(key) + i) % self.capacity     # linear probing

    def put(self, key, value):
        first_tomb = -1
        for i in range(self.capacity):
            idx = self._probe(key, i)
            if self.keys[idx] == EMPTY:
                slot = first_tomb if first_tomb != -1 else idx
                self.keys[slot] = key
                self.values[slot] = value
                self.size += 1
                if self.size / self.capacity > 0.7:
                    self._rehash()
                return
            if self.keys[idx] == TOMB:
                if first_tomb == -1:
                    first_tomb = idx
            elif self.keys[idx] == key:
                self.values[idx] = value                 # update existing
                return

    def get(self, key):
        for i in range(self.capacity):
            idx = self._probe(key, i)
            if self.keys[idx] == EMPTY:
                return -1
            if self.keys[idx] == key:
                return self.values[idx]
        return -1

    def remove(self, key):
        for i in range(self.capacity):
            idx = self._probe(key, i)
            if self.keys[idx] == EMPTY:
                return
            if self.keys[idx] == key:
                self.keys[idx] = TOMB
                self.size -= 1
                return

    def _rehash(self):
```

```
        old_keys = self.keys
        old_vals = self.values
        self.capacity *= 2
        self.keys = [EMPTY] * self.capacity
        self.values = [0] * self.capacity
        self.size = 0
        for j in range(len(old_keys)):
            if old_keys[j] != EMPTY and old_keys[j] != TOMB:
                self.put(old_keys[j], old_vals[j])

def main():
    lines = [ln for ln in sys.stdin.read().splitlines() if ln.strip()]
    cap = int(lines[0])
    m = MyHashMap(cap)
    out = []
    for line in lines[1:]:
        p = line.split()
        if p[0] == "PUT":
            m.put(int(p[1]), int(p[2]))
        elif p[0] == "GET":
            out.append(str(m.get(int(p[1]))))
        elif p[0] == "REMOVE":
            m.remove(int(p[1]))
    out.append("Size: " + str(m.size))
    out.append("Capacity: " + str(m.capacity))
    print("\n".join(out))

main()
```

## C++

```
// Lab 3 - MyHashMap with Open Addressing (linear probing + tombstones + resizing)
#include <iostream>
#include <string>
#include <sstream>
#include <vector>
using namespace std;
static const int EMPTY = -1, TOMB = -2;
class MyHashMap {
    int capacity, size;
    vector<int> keys, values;
    int hashKey(int key){ return key % capacity; }
    int probe(int key, int i){ return (hashKey(key) + i) % capacity; }
    void rehash(){
        vector<int> ok = keys, ov = values;
        capacity *= 2;
        keys.assign(capacity, EMPTY);
        values.assign(capacity, 0);
        size = 0;
        for (size_t j = 0; j < ok.size(); j++)
            if (ok[j] != EMPTY && ok[j] != TOMB) put(ok[j], ov[j]);
    }
public:
    MyHashMap(int cap): capacity(cap), size(0), keys(cap, EMPTY), values(cap, 0) {}
    void put(int key, int value){
        int firstTomb = -1;
        for (int i = 0; i < capacity; i++){
```

```
            int idx = probe(key, i);
            if (keys[idx] == EMPTY){
                int slot = (firstTomb != -1) ? firstTomb : idx;
                keys[slot] = key; values[slot] = value; size++;
                if ((double)size / capacity > 0.7) rehash();
                return;
            }
            if (keys[idx] == TOMB){ if (firstTomb == -1) firstTomb = idx; }
            else if (keys[idx] == key){ values[idx] = value; return; }
        }
    }
    int get(int key){
        for (int i = 0; i < capacity; i++){
            int idx = probe(key, i);
            if (keys[idx] == EMPTY) return -1;
            if (keys[idx] == key) return values[idx];
        }
        return -1;
    }
    void remove(int key){
        for (int i = 0; i < capacity; i++){
            int idx = probe(key, i);
            if (keys[idx] == EMPTY) return;
            if (keys[idx] == key){ keys[idx] = TOMB; size--; return; }
        }
    }
    int getSize(){ return size; }
    int getCapacity(){ return capacity; }
};
int main(){
    string line; getline(cin, line);
    int cap = stoi(line);
    MyHashMap m(cap);
    vector<string> out;
    while (getline(cin, line)){
        if (line.empty()) continue;
        istringstream iss(line); string op; iss >> op;
        if (op == "PUT"){ int k, v; iss >> k >> v; m.put(k, v); }
        else if (op == "GET"){ int k; iss >> k; out.push_back(to_string(m.get(k))); }
        else if (op == "REMOVE"){ int k; iss >> k; m.remove(k); }
    }
    out.push_back("Size: " + to_string(m.getSize()));
    out.push_back("Capacity: " + to_string(m.getCapacity()));
    for (auto& x : out) cout << x << "\n";
    return 0;
}
```

## Java

```
// Lab 3 - MyHashMap with Open Addressing (linear probing + tombstones + resizing)
import java.util.*;
public class Main {
    static final int EMPTY = -1, TOMB = -2;
    static class MyHashMap {
        int capacity, size = 0;
        int[] keys, values;
```

```
        MyHashMap(int cap){ capacity = cap; keys = new int[cap]; values = new int[cap];
Arrays.fill(keys, EMPTY); }
        int hashKey(int key){ return key % capacity; }
        int probe(int key, int i){ return (hashKey(key) + i) % capacity; }
        void put(int key, int value){
            int firstTomb = -1;
            for (int i = 0; i < capacity; i++){
                int idx = probe(key, i);
                if (keys[idx] == EMPTY){
                    int slot = (firstTomb != -1) ? firstTomb : idx;
                    keys[slot] = key; values[slot] = value; size++;
                    if ((double) size / capacity > 0.7) rehash();
                    return;
                }
                if (keys[idx] == TOMB){ if (firstTomb == -1) firstTomb = idx; }
                else if (keys[idx] == key){ values[idx] = value; return; }
            }
        }
        int get(int key){
            for (int i = 0; i < capacity; i++){
                int idx = probe(key, i);
                if (keys[idx] == EMPTY) return -1;
                if (keys[idx] == key) return values[idx];
            }
            return -1;
        }
        void remove(int key){
            for (int i = 0; i < capacity; i++){
                int idx = probe(key, i);
                if (keys[idx] == EMPTY) return;
                if (keys[idx] == key){ keys[idx] = TOMB; size--; return; }
            }
        }
        void rehash(){
            int[] ok = keys, ov = values;
            capacity *= 2;
            keys = new int[capacity]; values = new int[capacity]; Arrays.fill(keys, EMPTY);
            size = 0;
            for (int j = 0; j < ok.length; j++)
                if (ok[j] != EMPTY && ok[j] != TOMB) put(ok[j], ov[j]);
        }
    }
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int cap = Integer.parseInt(sc.nextLine().trim());
        MyHashMap m = new MyHashMap(cap);
        StringBuilder sb = new StringBuilder();
        while (sc.hasNextLine()){
            String line = sc.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            if (p[0].equals("PUT")) m.put(Integer.parseInt(p[1]), Integer.parseInt(p[2]));
            else if (p[0].equals("GET"))
sb.append(m.get(Integer.parseInt(p[1]))).append("\n");
            else if (p[0].equals("REMOVE")) m.remove(Integer.parseInt(p[1]));
        }
        sb.append("Size: ").append(m.size).append("\n");
        sb.append("Capacity: ").append(m.capacity).append("\n");
```

```
        System.out.print(sb);
    }
}
```

## Sample Run

### Sample Input

```
4
PUT 1 100
PUT 5 200
GET 1
GET 5
REMOVE 1
GET 1
PUT 1 300
GET 1
PUT 9 900
PUT 2 200
GET 9
GET 2
```

### Sample Output

```
100
200
-1
300
900
200
Size: 4
Capacity: 8
```

## Follow-Up & Discussion

- Tombstones let a search keep probing past a deleted slot to find keys placed later in the same chain; without them, deleting a middle entry by emptying its slot would make those later keys unreachable.
- Resizing under the hood: C++ std::unordered_map rehashes its buckets when the load factor exceeds max_load_factor (it chains, so it grows a prime-sized bucket array); Java HashMap doubles its power-of-two table at the 0.75 threshold and re-distributes (treeifying long buckets); Python dict grows (roughly x4 when small) and re-inserts into a new open-addressed table.
- Chaining versus open addressing in production: chaining tolerates high load factors and adversarial inputs more gracefully and deletes simply; open addressing is faster and more memory-compact at moderate load thanks to cache locality - the right pick depends on load, key distribution and deletion frequency.
- Cuckoo or Hopscotch hashing become attractive when you need worst-case O(1) lookups (not just average): cuckoo guarantees at most two probe locations per key, and hopscotch

keeps each key within a bounded neighbourhood of its home slot - both bound the probe cost that linear probing leaves unbounded.